HotelPro – Functional Specification Document

1. Introduction

- **Project title: HotelPro – Complete Hotel Booking and Management Solution (MERN-style web application)**
- **Team members: SWUID2026403439**

2. Project Overview

Purpose. HotelPro lets guests discover hotels, compare room types and prices, check availability for selected dates, and complete reservations online. Hotel administrators manage hotel listings, rooms, rates, inventory and bookings through a separate admin panel. This local-development version stores application data in JSON files rather than a database.

Guest features

- Register, log in, edit profile and change password
- Browse, search and filter hotels by city, dates, guest count, amenities, price range and rating, with sorting
- Hotel detail page with photos, address, amenities, policies, room types, rates and availability
- Room selection with live-in-session availability checks, guest details and booking summary
- Simulated payment by card, UPI, net banking or wallet
- Booking confirmation with booking ID, printable voucher, booking history and cancellation according to the hotel's policy, with simulated refund status
- Star ratings and reviews; save favourite hotels and manage a wishlist

Admin features (at /admin/login)

- Dashboard: booking revenue, reservations, occupancy, popular hotels and recent activity
- Manage hotel listings, room types, amenities, photos and property policies
- Manage room rates, date-based availability, inventory and booking restrictions
- View, update and cancel reservations; activate, deactivate or remove user accounts

3. Architecture

Frontend (React + Vite). A single-page app in JavaScript/JSX using React Router. App.jsx defines routes; RequireAuth guards guest pages (role="user") and the admin area (role="admin"). Login state lives in AuthContext, API calls go through api.js, and shared UI sits in components/ (Navbar, Footer, HotelCard, RoomCard and shared UI elements). The Vite development server proxies /api to port 5000 so the frontend and API work together locally.

Backend (Node.js + Express). server.js configures the app (security headers, JSON parsing, cookie handling, request logging and rate limiting on login/register) and mounts route groups under /api for auth, hotels, rooms, bookings, payments, reviews, favourites and admin. Controllers hold business logic; middleware handles authentication, role checks, input validation and centralised

errors. utils/ contains booking-code generation, availability helpers, simulated payment processing, optional email sending and JSON-file storage helpers.

Local data storage (JSON files). Application records are stored in server/data/ and read or written by storage utilities. Files include users.json, admins.json, hotels.json, rooms.json, bookings.json, reviews.json and favourites.json. This setup is intended for local development and a single running backend process; it is not a multi-server production database.

Data file / entity	Key fields	Relations
User (users.json)	id, name, email (unique), hashed password, phone, isActive	Bookings, reviews, favourites
Admin (admins.json)	id, name, email (unique), hashed password	Admin access
Hotel (hotels.json)	name, description, city, address, amenities[], photos[], rating, policies, isActive	Rooms and bookings
Room (rooms.json)	hotelId, roomType, capacity, bedType, amenities[], pricePerNight, inventory, isActive	Hotel
Booking (bookings.json)	bookingCode, checkIn, checkOut, guests, roomId, totalAmount, status, cancellationPolicy	User, hotel, room, payment
Payment (stored with booking)	method, status, transactionId, paidAt, refundStatus	Booking
Review (reviews.json)	rating 1–5, comment (max 1000 chars), createdAt	User, hotel
Favourite (favourites.json)	type (favourite / wishlist), unique user + hotel + type	User, hotel

4. Setup Instructions

Prerequisites

- Node.js 18 or newer (check with node -v)
- No database server is required. The application stores data in local JSON files under server/data/.

Installation

1. Extract the project and open two terminals, one in server/ and one in client/.
2. In server/, run npm install.
3. Create server/.env from the example file and set the application variables below (change JWT_SECRET to a long random string).
4. Run npm run seed, if provided, to create a demo admin account and sample hotels, rooms and bookings in local JSON files.
5. In client/, run npm install.

Variable (server/.env)	Purpose
PORT	API port (default 5000)
NODE_ENV	development or production
JWT_SECRET	Secret used to sign tokens
JWT_EXPIRES_IN	Token lifetime (default 7d)
CLIENT_URL	Allowed CORS origin (default http://localhost:5173)
BOOKING_HOLD_MINUTES	Optional reservation hold duration while payment is pending
ADMIN_NAME, ADMIN_EMAIL, ADMIN_PASSWORD	Default admin created by seed script, if configured
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, EMAIL_FROM	Optional email settings; if not configured, messages print in server console
DATA_DIR	Optional path to local JSON data files (default server/data/)

Optional in client/: VITE_API_URL is needed only when the API is hosted on a different origin.

5. Folder Structure

Client (client/)

client/\n index.html, vite.config.js, package.json, .env.example\n public/favicon.svg\n src/\n main.jsx, App.jsx entry point and routes\n api.js, hooks.js, utils.js\n index.css\n context/AuthContext.jsx login state\n components/ Navbar, Footer, HotelCard, RoomCard, shared UI\n pages/ Home, Hotels, HotelDetail, RoomSelection, Checkout,\n BookingDetail, MyBookings, Favourites, Profile, Login, Register\n pages/admin/ AdminLogin, AdminLayout, Dashboard, AdminHotels,\n AdminRooms, AdminBookings, AdminUsers

Server (server/)

server/\n index.js entry point: loads .env and starts the server\n server.js Express app, middleware and route mounting\n data/ users.json, admins.json, hotels.json, rooms.json, bookings.json, reviews.json, favourites.json\n models/ data shape / validation helpers (if used)\n controllers/ auth, hotel, room, booking, payment, review, favourite, admin\n routes/ one router per feature group\n middleware/ authMiddleware, role checks, validation and error handling\n utils/ jsonStore, availability, bookingCode, paymentGateway, sendEmail, seed, AppError

6. Running the Application

Start the backend first, then the frontend, each in its own terminal.

Part	Directory	Command	Address
Backend (development)	server/	npm run dev (nodemon)	http://localhost:5000
Backend (plain)	server/	npm start	http://localhost:5000
Frontend (development)	client/	npm run dev	http://localhost:5173

The project uses Vite, so the frontend command is npm run dev rather than npm start. Check the API at /api/health. JSON data files must be writable by the backend. For production, run npm run build in client/ and serve client/dist from a static host; replace local JSON storage with a suitable persistent database before multi-user production deployment.

Role	Login URL	Credentials
Guest	/register	Any email and password you create
Admin	/admin/login	Use configured seeded demo credentials; change before deployment

7. API Documentation

Base path /api. Every response is JSON with a success flag; errors return { "success": false, "message": "..." }. Access: Public = no login, User = guest login, Admin = admin login.

Method	Endpoint	Access	Purpose / parameters
GET	/health	Public	API status check
POST	/auth/register	Public	Create account: name, email, password, phone
POST	/auth/login	Public	Log in: email, password
POST	/auth/logout	Public	Clear the login cookie
GET	/auth/me	Any	Current account
PUT	/auth/me	Any	Update profile / change password
POST	/admin/login	Public	Admin login: email, password
GET	/admin/dashboard	Admin	Revenue, reservations, occupancy and recent activity
GET	/admin/users	Admin	List users
GET	/admin/users/:id	Admin	View one user
PUT, DELETE	/admin/users/:id	Admin	Activate/deactivate or remove a user

Method	Endpoint	Access	Purpose / parameters
GET	/admin/bookings	Admin	List and filter all reservations
GET	/hotels	Public	Query: city, checkIn, checkOut, guests, amenities, minPrice, maxPrice, rating, sort, page, limit
GET	/hotels/:id	Public	Hotel details, policies, amenities and room types
POST, PUT, DELETE	/hotels, /hotels/:id	Admin	Create, update or remove a hotel
GET	/rooms	Public	Query: hotelId, checkIn, checkOut, guests
GET	/rooms/:id	Public	Room details and pricing
POST, PUT, DELETE	/rooms, /rooms/:id	Admin	Manage room types, rates and inventory
GET	/availability	Public	Check hotel/room availability for date range and guest count
GET	/bookings/my	User	Current guest's reservation history
POST	/bookings	User	Create pending reservation: roomId, checkIn, checkOut, guests
POST	/bookings/:id/pay	User	Simulated payment: method and payment details
GET	/bookings/:id	Any	Reservation details (owner or admin)
PUT	/bookings/:id/cancel	Any	Cancel according to property's policy
PUT	/bookings/:id/status	Admin	Update reservation status where permitted
GET	/reviews/hotel/:hotelId	Public	Reviews for a hotel
POST	/reviews	User	Add review: hotelId, rating (1–5), comment
PUT	/reviews/:id	User	Edit own review
DELETE	/reviews/:id	Any	Delete review (owner or admin)
GET	/favourites	User	List saved hotels; query type=favourite/wishlist
POST	/favourites	User	Save hotel: hotelId, type

Method	Endpoint	Access	Purpose / parameters
DELETE	/favourites/:hotelId	User	Remove saved hotel; query type
POST	/admin/seed	Admin	Optional local demo-data seeding endpoint, if implemented

Example: login

```
POST /api/auth/login\n{ "email": "guest@example.com", "password": "secret123" }\n\n200 OK\n{\n  "success": true,\n  "token": "<jwt>",\n  "user": { "id": "...", "name": "Asha", "email": "guest@example.com", "role": "user", "phone": "..." }\n}
```

Example: cancel a reservation

```
PUT /api/bookings/:id/cancel\n\n200 OK\n{ "success": true, "message": "Cancellation requested; refund status updated where applicable.", "booking": { ... } }\n\n400 Bad Request (outside the property's cancellation policy)
```

Booking flow. POST /bookings validates the selected room and dates, calculates the stay total and creates a pending reservation. POST /bookings/:id/pay records the simulated payment and confirms the reservation. Availability is rechecked before confirmation. Actual payment processing and refunds are not connected.

8. Authentication

- **Passwords are hashed with bcrypt before saving and are excluded from queries by default.**
- **Tokens. On login or registration the server signs a JWT containing the account id and role (user or admin), valid for 7 days by default (JWT_EXPIRES_IN).**
- **Cookie. The token is sent in an httpOnly cookie named token (secure and sameSite settings depend on environment), so scripts in the page cannot read it. An Authorization: Bearer <token> header may also be accepted.**
- **Authorization. protect verifies the token and loads the account, rejecting deleted or deactivated users; adminOnly and userOnly enforce roles. Admins have their own login endpoint.**
- **Frontend. AuthContext loads the account from /auth/me, and RequireAuth redirects visitors away from pages that need a login or a specific role.**
- **Hardening. Use security headers, restrict CORS to CLIENT_URL, validate incoming data and rate-limit login/register attempts. Store secrets in environment variables.**

9. User Interface

Screens are listed with their routes. Screenshots or GIFs can be added under each row once the HotelPro application is running.

Screen	Route	Access	What it shows
Home	/	Public	Search hotels by destination, dates and guests; featured properties
Hotels	/hotels	Public	Hotel cards, filters, sorting and availability
Hotel detail	/hotels/:id	Public	Photos, location, amenities, policies, room types and rates

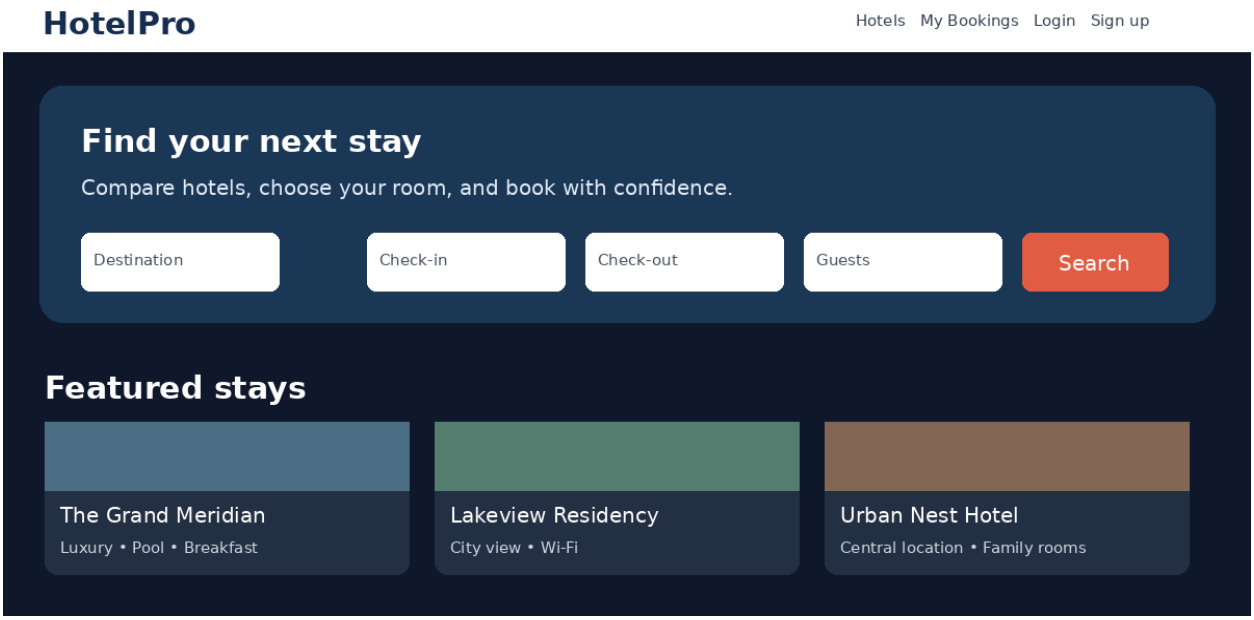
Screen	Route	Access	What it shows
Room selection	/hotels/:id/rooms	Public	Available room types and stay pricing for selected dates
Checkout	/checkout/:id	User	Guest details, booking summary and simulated payment
Booking detail	/bookings/:id	Login	Reservation voucher, booking ID, stay dates and status
My bookings	/my-bookings	User	Reservation history and cancellation options
Favourites	/favourites	User	Saved hotels and wishlist
Profile	/profile	User	Edit personal details and change password
Login / Register	/login, /register	Public	Guest authentication
Admin login	/admin/login	Public	Administrator authentication
Admin panel	/admin	Admin	Dashboard, hotels, rooms, availability, bookings and users

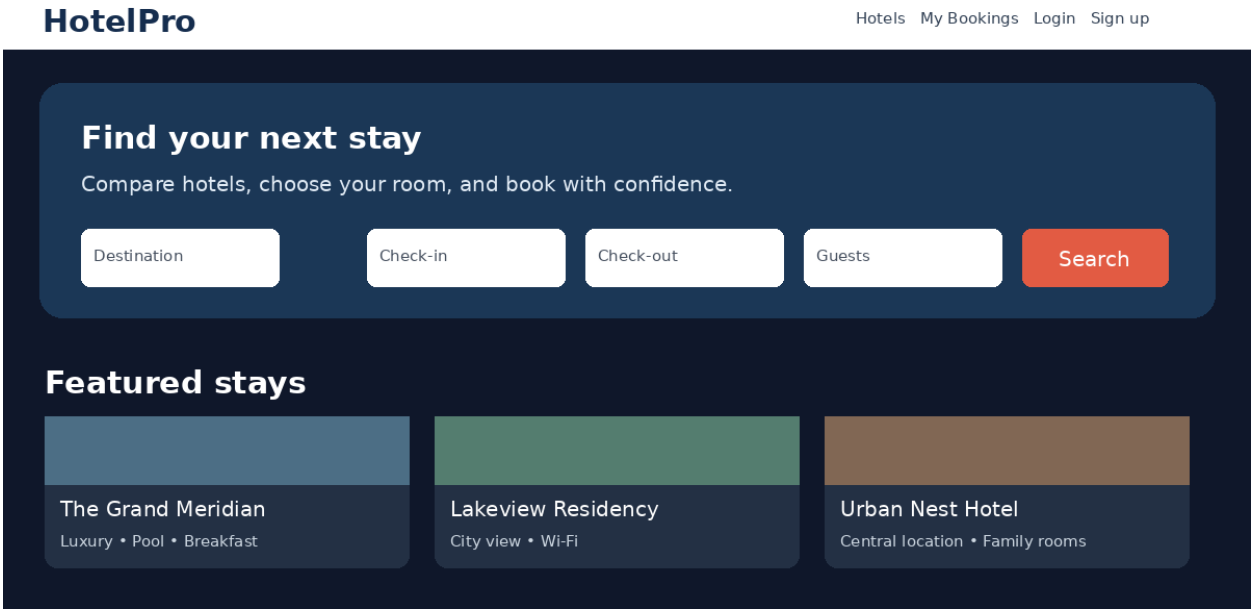
10. Testing

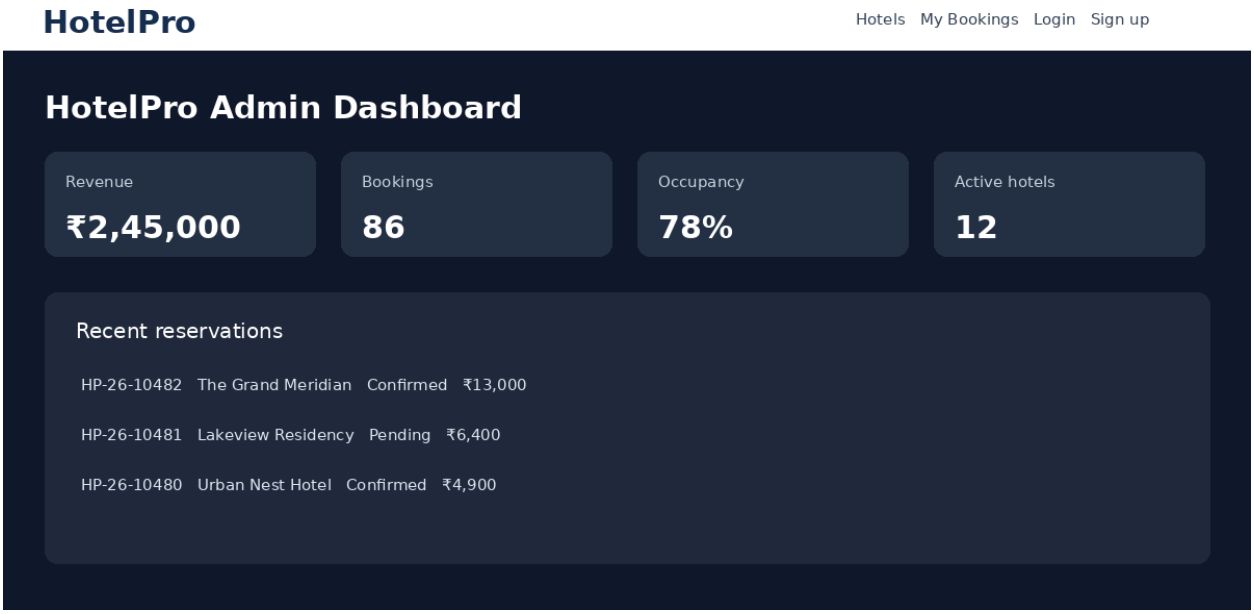
The project should be verified against the main guest and administrator flows. The checks below are recommended for the local build:

- **Manual end-to-end testing of registration, login, hotel search, date selection, room availability, reservation, simulated payment, voucher display, cancellation, reviews and favourites, plus every admin screen.**
- **Seeded demo data (npm run seed, if configured) provides a repeatable starting state.**
- **Payment test cases should verify accepted and rejected demo inputs, successful simulated payment, duplicate submission prevention and correct booking status updates.**
- **API checks with Postman or curl against /api/health and the endpoints in section 7.**
- **ESLint (npm run lint in client/, if configured) for static checks.**

11. Screenshots or Demo







HotelPro
Hotels
My Bookings
Login
Sign up
HotelPro Admin Dashboard
Revenue
₹2,45,000
Bookings
86
Occupancy
78%
Active hotels
12
Recent reservations
HP-26-10482 The Grand Meridian Confirmed ₹13,000
HP-26-10481 Lakeview Residency Pending ₹6,400
HP-26-10480 Urban Nest Hotel Confirmed ₹4,900

HotelPro

Hotels My Bookings Login Sign up

Hotels in your destination

Filters

Price range

Guest rating

Amenities

Breakfast

Free Wi-Fi

Pool

Parking



The Grand Meridian

★ 4.6 • Free Wi-Fi • Breakfast available

₹6,500 / night

View rooms



Lakeview Residency

★ 4.6 • Free Wi-Fi • Breakfast available

₹3,200 / night

View rooms



Urban Nest Hotel

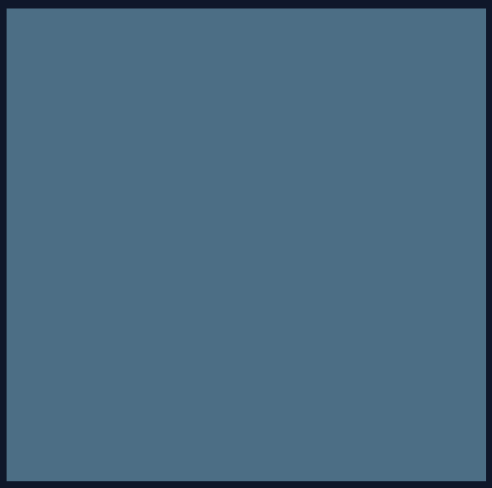
★ 4.6 • Free Wi-Fi • Breakfast available

₹2,450 / night

View rooms

HotelPro

Hotels My Bookings Login Sign up



The Grand Meridian

★ 4.8 • Downtown, Indore

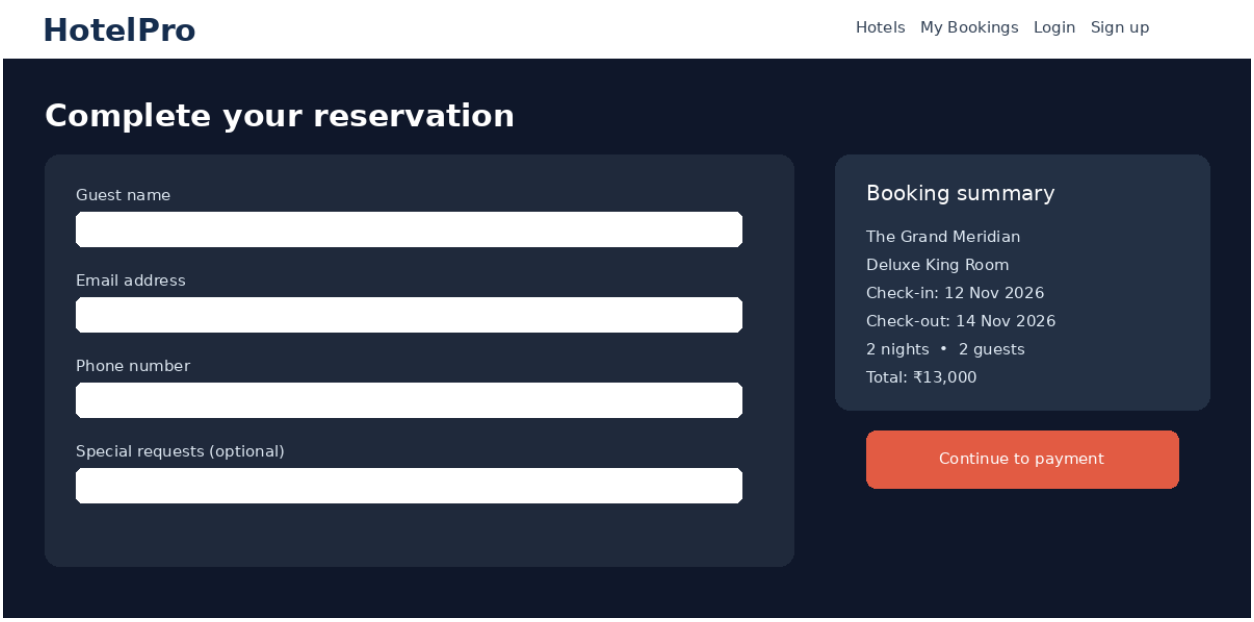
Comfortable rooms, attentive service and modern amenities.

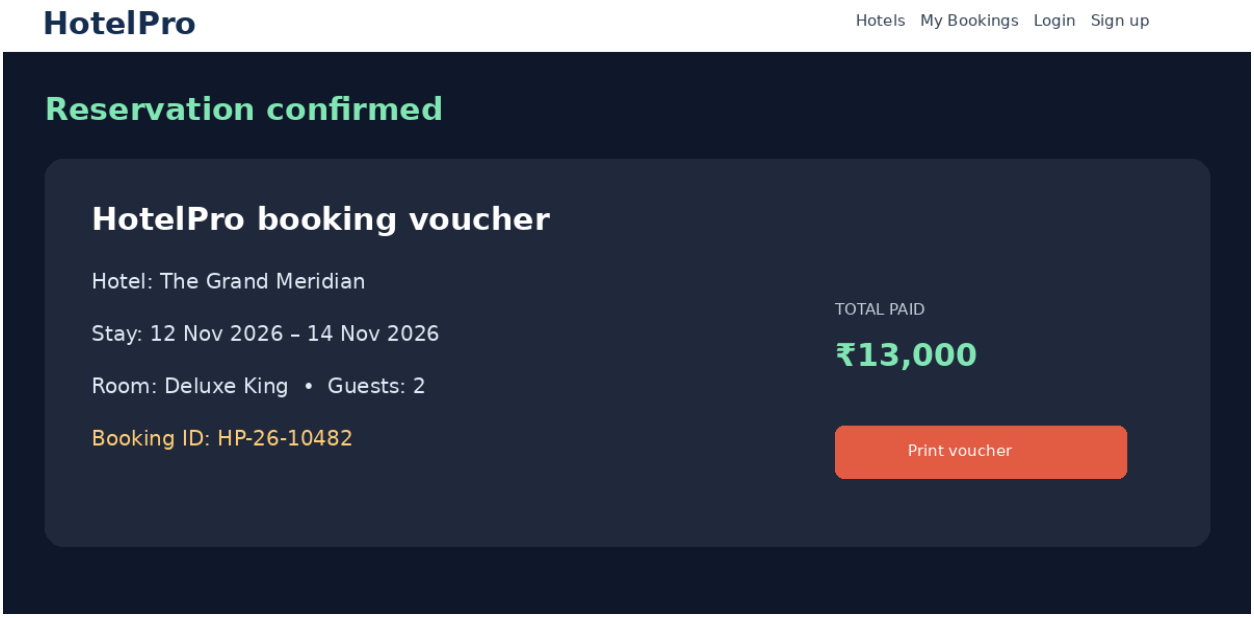
Amenities

Wi-Fi • Breakfast • Pool • Parking • Air conditioning

Available room types

Deluxe King	₹6,500/night
Twin Room	₹5,900/night





12. Known Issues

- Payments are simulated: no real payment gateway is connected and refunds are only recorded as a status.
- Emails are printed to the server console unless SMTP settings are configured.
- Hotel, room and availability records are local sample data and must be updated to reflect actual properties and dates.
- Port 5000 may already be in use; change PORT in server/.env and the proxy target in client/vite.config.js together.
- The seeded admin credentials are for demonstration only and must be changed before any deployment.

- Local JSON files are suitable for a single local instance; concurrent writes, backups and multi-server deployment require stronger persistence and locking.

13. Future Enhancements

- Integrate a real payment gateway with verified payment status and refunds
- Automated unit, integration and API tests, and CI
- Email/SMS reservation vouchers with QR codes and reminders
- Image upload and storage for hotel and room photos
- Coupons, seasonal pricing, taxes, add-ons and special packages
- Personalised hotel recommendations based on searches, favourites and past bookings
- Multi-language interface, currency options and a mobile application